Université Ibn Zohr
Ecole Supérieure de Technologie – Guelmim
Département d'Informatique

A.U. 2025-2026
DUT ID/S3

TP 2 : Algorithmes de Tri

TRI PAR SÉLECTION

Exercice 1.1 - Implémentation

```python
def tri_selection(liste):
    n = len(liste)
    for i in range(n):
        # À COMPLÉTER
    return liste
```

Énoncé : Complétez la fonction tri_selection et écrivez un programme principal qui trie la liste [64, 25, 12, 22, 11]

Exercice 1.2 - Tri décroissant

Énoncé : Modifiez l'algorithme pour trier dans l'ordre **décroissant** et testez avec [5, 2, 8, 1, 9]

TRI PAR INSERTION

Exercice 2.1 - Implémentation avec affichage

Énoncé : Écrivez une fonction tri_insertion_affichage(liste) qui :

- Trie par insertion
- Affiche la liste **après chaque insertion complète**
- Testez avec [31, 41, 59, 26, 41]

Exercice 2.2 - Insertion avec sentinelle

Énoncé : Implémentez le tri par insertion en ajoutant une **sentinelle** (premier élément comme valeur minimale) pour optimiser les comparaisons.

TRI À BULLES

Exercice 3.1 - Version de base

```python
def tri_bulles(liste):
    # À ÉCRIRE
    return liste
```

Énoncé : Implémentez le tri à bulles et testez avec [5, 1, 4, 2, 8]

Exercice 3.2 - Version optimisée

Énoncé : Améliorez l'algorithme pour qu'il s'arrête si **aucun échange** n'est effectué lors d'un passage.

TRI RAPIDE

Exercice 4.1 - Partitionnement

```
def partitionner(liste, debut, fin):
    # À COMPLÉTER
    return indice_pivot

def tri_rapide(liste, debut, fin):
    # À COMPLÉTER
    return liste
```

Énoncé : Complétez les fonctions en utilisant le **premier élément** comme pivot. Testez avec [10, 80, 30, 90, 40]

Exercice 4.2 - Pivot milieu

Énoncé : Modifiez l'algorithme pour utiliser l'élément du **milieu** comme pivot. Testez avec

[7, 2, 1, 6, 8, 5]

TRI FUSION

Exercice 5.1 - Fusion de listes

Vous devez trier une liste d'étudiants selon leurs notes en utilisant l'algorithme de tri par fusion. Chaque étudiant est représenté par un tuple (nom, note).

Liste des étudiants :

[("Ahmed", 15), ("Maryem", 12), ("Anas", 18), ("Safae", 14), ("Ali", 16), ("Fatima", 13)]

```
Entrée [ ]: def tri_fusion_etudiants(etudiants):
                #  À COMPLÉTER

                return etudiants

            def fusionner_etudiants(gauche, droite):
                resultat = []
                i = j = 0

                # À COMPLÉTER
                # Fusionner les deux listes en comparant les notes
                # Si gauche[i][1] <= droite[j][1], ajouter gauche[i]
                # Sinon ajouter droite[j]

                return resultat
```

```
# Liste des étudiants
etudiants = [("Ahmed", 15), ("Maryem", 12), ("Anas", 18), ("Safae", 14), ("Ali", 16), ("Fatima", 13)]

print("Liste originale des étudiants:")
for nom, note in etudiants:
    print(f"- {nom}: {note}/20")

# Tri par fusion
etudiants_tries = tri_fusion_etudiants(etudiants)

print("\nListe triée par notes (croissant):")
for nom, note in etudiants_tries:
    print(f"- {nom}: {note}/20")
```